Antigravity Drift Dashboard Prompt

- Antigravity IDE — Full Development Prompt
  - Linux Infrastructure Drift Detection & Auto Repair System — Professional Web Dashboard
    - ROLE
  - PHASE 0: AGENT OPERATING RULES (Antigravity-specific)
  - PHASE 1: INSPECT THE EXISTING PROJECT
  - PHASE 2: TECHNOLOGY STACK
  - PHASE 3: PROJECT ARCHITECTURE
  - PHASE 4: PROFESSIONAL UI DESIGN
    - Visual style
    - Main layout
  - PHASE 5: DASHBOARD OVERVIEW
    - Summary cards
    - Compliance chart
    - Recent drift
    - Recent activity
    - Quick actions
  - PHASE 6: CONFIGURATION PAGE
  - PHASE 7: DRIFT DETECTION
  - PHASE 8: SIMULATE DRIFT
  - PHASE 9: AUTO REPAIR
    - Repair eligibility (allowlist)
    - Repair preview
    - Repair process
    - Repair results
  - PHASE 10: BACKUP MANAGEMENT
  - PHASE 11: REPORTS PAGE
  - PHASE 12: AUDIT LOGS
  - PHASE 13: FLASK API
    - Security hardening
  - PHASE 14: RESPONSIVE DESIGN
  - PHASE 15: TESTING AND VERIFICATION
  - PHASE 16: INSTALLATION AND RUNNING
  - PHASE 17: FINAL DELIVERABLES
  - FINAL EXECUTION INSTRUCTIONS

Antigravity IDE — Full Development Prompt

Linux Infrastructure Drift Detection & Auto Repair System — Professional Web Dashboard

ROLE

Act as a senior Python developer, Flask backend engineer, frontend developer, UI/UX designer, Linux system administrator, and cybersecurity engineer.

I have an existing project called **Linux Infrastructure Drift Detection & Auto Repair System**. I want you to develop a complete, professional, responsive graphical web dashboard integrated with my existing Python application.

Work directly inside the currently opened project in Antigravity IDE.

IMPORTANT: Do not create a separate unrelated project. Do not delete or break my existing CLI. Inspect the current code before modifying anything.

PHASE 0: AGENT OPERATING RULES (Antigravity-specific)

- Work in small, verifiable increments. After each phase, run the test suite and report pass/fail before moving to the next phase — do not batch all changes into one giant unreviewed commit.
- Before writing any new file, check whether an equivalent file already exists in the project and update it instead of duplicating it.
- Keep a running changelog (`CHANGELOG.md` or equivalent) of every file created or modified, with a one-line reason for each.
- Do not install packages from the internet without listing them first. If the sandbox has no internet access, vendor Chart.js and the icon library as local static files under `web_dashboard/static/vendor/` instead of loading them from a CDN, so the dashboard still works offline. Prefer this local-vendoring approach by default; only use a CDN if explicitly confirmed the environment has outbound internet access.
- Never commit secrets, `.env` files, `backups/`, `reports/`, or `logs/audit.log` contents to version control. Add/extend a `.gitignore` to cover them.

PHASE 1: INSPECT THE EXISTING PROJECT

First, inspect the complete project structure and read the existing Python files.

The project contains or may contain:

- `baseline.json`
- `current_config.json`
- `drift_detector.py`
- `linux_repair.py`
- `report_generator.py`
- `tests/test_project.py`
- `tests/test_backup.py`
- `backups/`
- `reports/`
- `logs/audit.log`

My existing CLI provides:

1. View current configuration.
2. Detect configuration drift.
3. Simulate configuration drift.
4. Automatically repair detected drift.
5. View audit logs.
6. Exit.

Before changing files:

1. Inspect the existing implementation.
2. Understand how configuration comparison works.
3. Identify the existing simulation, repair, backup, reporting and logging functions.
4. Run the current test suite.
5. Record the test results.
6. Prepare an implementation plan.
7. Preserve all working functionality.

Do not assume functions exist without inspecting the code.

If a required function is missing, implement it safely and document the change.

PHASE 2: TECHNOLOGY STACK

Use this stack:

Backend: - Python 3 (match the version already in use in the project; confirm via `python3 --version` rather than assuming) - Flask - REST-style JSON API - Existing Python modules

Frontend: - HTML5 - CSS3 - JavaScript - Fetch API - Chart.js for visualizations if useful (vendored locally — see Phase 0) - Lucide icons or another consistent icon library (vendored locally — see Phase 0)

Storage: - Existing JSON configuration files - Existing backups directory - Existing reports directory - Existing audit logs

Testing: - Existing Python unittest suite - Additional API and integration tests

Use a Python virtual environment if one already exists. Otherwise, create one without overwriting existing environment files.

Produce and maintain a pinned `requirements.txt` (exact versions, e.g. `Flask==3.0.3`) so the environment is reproducible. Do not leave dependency versions unpinned.

Avoid unnecessary dependencies and databases.

PHASE 3: PROJECT ARCHITECTURE

Create a clean structure similar to this, adapting it to the existing project rather than blindly replacing files:

```
linux-drift-detection/
├── baseline.json
├── current_config.json
├── drift_detector.py
├── linux_repair.py
├── report_generator.py
├── requirements.txt
├── .gitignore
│
├── core/
│   ├── __init__.py
│   └── drift_service.py
│
├── web_dashboard/
│   ├── app.py
│   ├── templates/
│   │   ├── base.html
│   │   ├── index.html
│   │   ├── configuration.html
│   │   ├── drift_detection.html
│   │   ├── simulate_drift.html
│   │   ├── auto_repair.html
│   │   ├── backups.html
│   │   ├── reports.html
│   │   ├── audit_logs.html
│   │   └── settings.html
│   └── static/
│       ├── css/
│       │   └── style.css
│       ├── js/
│       │   ├── app.js
│       │   ├── dashboard.js
│       │   ├── configuration.js
│       │   ├── drift.js
│       │   ├── repair.js
│       │   └── logs.js
│       ├── vendor/
│       │   ├── chart.js
│       │   └── lucide.js
│       └── assets/
│
├── tests/
├── backups/
├── reports/
└── logs/
```

The CLI, web dashboard and Python service layer must share the same business logic.

Do not implement separate drift detection or repair algorithms in JavaScript.

The backend is the source of truth.

PHASE 4: PROFESSIONAL UI DESIGN

Build a polished, professional infrastructure monitoring dashboard inspired by modern DevOps, cloud monitoring and security compliance products.

Visual style

Use a dark theme by default.

Suggested colors: - Background: `#0B1120` - Sidebar: `#111827` - Cards: `#172033` - Primary blue: `#3B82F6` - Success green: `#22C55E` - Warning amber: `#F59E0B` - Error red: `#EF4444` - Primary text: `#F8FAFC` - Secondary text: `#94A3B8` - Borders: `#293548`

Use: - Professional typography. - Consistent spacing. - Rounded cards. - Subtle borders and shadows. - Clear visual hierarchy. - Consistent iconography. - Responsive tables. - Useful charts. - Status badges. - Tooltips. - Loading skeletons or spinners. - Success and error notifications. - Empty states. - Confirmation dialogs.

Do not make the entire dashboard red. Reserve red for critical errors and failed operations.

Avoid a generic template look. Make the interface feel like a real infrastructure administration product.

Main layout

Create a persistent sidebar with:

1. Dashboard
2. Configuration
3. Drift Detection
4. Simulate Drift
5. Auto Repair
6. Backups
7. Reports
8. Audit Logs
9. Settings

Create a top navigation bar with: - Current page title. - System compliance status. - Last scan timestamp. - Refresh action. - Theme toggle. - Backend connection status.

Ensure the active navigation item is highlighted.

PHASE 5: DASHBOARD OVERVIEW

Build a dashboard showing actual information from the Python backend.

Summary cards

Display:

1. System Compliance
2. Total Configuration Properties
3. Detected Drift
4. Successful Repairs
5. Available Backups
6. Generated Reports

Only show values that can be calculated from actual project data or audit records.

If a metric cannot be reliably calculated, display `Unavailable` or `Not recorded`.

Do not fabricate metrics or use hardcoded sample values in the final implementation.

Compliance chart

Create a chart displaying: - Compliant properties. - Drifted properties. - Invalid or unknown properties, where supported.

Recent drift

Display a table with: - Configuration key. - Expected value. - Actual value. - Status. - Severity if supported. - Detection time if available.

Recent activity

Display actual audit events: - Scan completed. - Drift detected. - Simulation executed. - Backup created. - Repair attempted. - Repair completed. - Repair failed. - Verification completed.

Quick actions

Add functional buttons: - Scan for Drift - View Configuration - Simulate Drift - Repair Drift - View Backups - Generate Report

All actions must call real backend functionality.

PHASE 6: CONFIGURATION PAGE

Read `baseline.json` and `current_config.json` through the Python backend.

Display a configuration comparison table with:

- Property name.
- Baseline value.
- Current value.
- Compliance status.
- Optional severity if defined.

Use visual status indicators: - Green for matching values. - Amber or red for detected differences. - Gray for unknown or unavailable values.

Support: - Search by property name. - Filter compliant properties. - Filter drifted properties. - Refresh configuration. - View details for an individual property.

Handle strings, booleans, numbers, null values and missing properties correctly.

Do not confuse a false value with a missing property.

Make the baseline read-only from the dashboard by default.

Do not allow unrestricted editing of configuration data.

PHASE 7: DRIFT DETECTION

Create a dedicated drift detection page.

Add a prominent `Scan for Drift` button.

When clicked:

1. Send a request to the Flask API.
2. Run the existing Python drift detector.
3. Compare the current configuration with the baseline.
4. Return actual detected differences.
5. Update the UI.
6. Refresh dashboard metrics.
7. Record the event through the existing audit logging system if supported.

Display: - Total properties checked. - Compliant properties. - Drifted properties. - Scan result. - Scan timestamp. - Expected and actual values.

Add loading indicators and clear error handling.

Do not claim a scan succeeded if the backend returned an error.

Do not label a property insecure merely because it differs from the baseline. Use the baseline comparison to establish drift; only assign security severity if explicit rules exist.

PHASE 8: SIMULATE DRIFT

Create a simulation page using the existing Python simulation functionality.

Provide: - Configuration property selector. - Current value. - Baseline value. - Proposed simulated value. - Simulate Drift button. - Cancel button.

When a simulation is started:

1. Validate the property.
2. Validate the proposed value.
3. Require confirmation if the project's current configuration will be modified.
4. Create a backup before modification where supported.
5. Execute the existing simulation logic.
6. Record the event in the audit log.
7. Run drift detection.
8. Show the result.

Clearly label the action as a simulation.

Do not modify actual Linux SSH settings, firewall rules, operating system services or system configuration during simulation.

Simulation should only modify the project's test configuration file.

PHASE 9: AUTO REPAIR

Build a professional repair page with a repair preview and explicit confirmation.

Repair eligibility (allowlist)

Define an explicit, centrally maintained allowlist of configuration keys that are eligible for automatic repair (e.g. a `REPAIRABLE_KEYS` set in the service layer). A property is "permitted" for repair only if: - it exists in both `baseline.json` and `current_config.json`, - it is present in the allowlist, and - its baseline value is itself valid (passes schema/type validation).

Any drifted property not on the allowlist must be reported as drift but marked **not eligible for automatic repair**, with a reason shown in the UI.

Repair preview

Before repair, show: - Drifted property. - Current value. - Expected baseline value. - Proposed replacement value. - Whether the property is eligible for repair (per the allowlist above).

Repair process

When the user confirms:

1. Revalidate the requested operation on the backend.
2. Re-read the latest configuration.
3. Recheck the current drift.
4. Create a backup before modifying anything.
5. Verify that the backup was created successfully.
6. Apply only permitted baseline values (per the allowlist).
7. Validate the resulting JSON.
8. Re-run drift detection.
9. Record the result in the audit log.
10. Return the actual result to the UI.

If backup creation fails, do not continue with the repair.

Use safe file-writing techniques (write to a temp file, then atomic rename) and appropriate concurrency controls (e.g. a file lock) so two concurrent repair requests cannot interleave.

Repair results

Show: - Repair status. - Number of properties repaired. - Remaining drift. - Verification result. - Backup information. - Error details if the operation fails.

Display `COMPLIANT` only when verification succeeds and the completed checks confirm that no drift remains.

Require confirmation before repair.

Do not trigger repairs when the user merely opens the page.

For this first version, repair only the project’s JSON configuration. Do not modify real operating system security settings.

PHASE 10: BACKUP MANAGEMENT

Build a backups page that reads the existing `backups/` directory.

Display: - Filename. - Timestamp. - File size. - Validity status. - Available actions.

Features: - Refresh backup list. - View backup details. - Compare backup with current configuration if supported. - Restore a backup after confirmation.

Before restore: 1. Validate the backup identifier. 2. Confirm the file is inside the approved backup directory (resolve the path and check it is a descendant of `backups/`; reject anything else). 3. Validate its JSON structure. 4. Back up the current configuration. 5. Apply the restore safely (temp file + atomic rename). 6. Verify the restored configuration. 7. Record the operation.

Prevent path traversal.

Never delete backups automatically.

PHASE 11: REPORTS PAGE

Build a reports page connected to the existing `reports/` directory and `report_generator.py`.

Display: - Report filename. - Timestamp if available. - File size. - Report format. - Availability status.

Provide: - Refresh reports. - View supported JSON reports. - Download approved reports. - Generate a new report.

Use the existing report generator wherever possible.

Do not claim PDF export is supported unless it is actually implemented.

Prevent users from downloading arbitrary files outside the reports directory (same path-resolution check as Phase 10).

PHASE 12: AUDIT LOGS

Build an audit logs page connected to `logs/audit.log` or the existing audit logging mechanism.

Display: - Timestamp. - Event type. - Description. - Operation status when available.

Support: - Search. - Event filtering. - Success/failure filtering when recorded. - Newest-first display when timestamps permit. - Pagination or bounded loading. - Refresh.

Include actual events for: - Startup. - Drift scan. - Drift simulation. - Backup creation. - Repair attempt. - Repair success. - Repair failure. - Restore attempt. - Post-repair verification.

Preserve existing log compatibility.

Audit logs must be read-only in the dashboard. Do not provide an interface to edit or delete them.

Use Python's standard `RotatingFileHandler` (or equivalent) for `audit.log` so it does not grow unbounded; do not change the existing log format, only the rotation policy.

PHASE 13: FLASK API

Implement a Flask backend with endpoints similar to:

- `GET /api/health`
- `GET /api/dashboard`
- `GET /api/configuration`
- `POST /api/drift/scan`
- `POST /api/drift/simulate`
- `POST /api/repair/preview`
- `POST /api/repair`
- `GET /api/backups`
- `POST /api/backups/<backup_id>/restore`
- `GET /api/reports`
- `GET /api/audit-logs`

Adapt the endpoints to the existing codebase.

Use consistent JSON response structures and proper HTTP status codes.

Validate every incoming request, including: - Correct `Content-Type: application/json` on POST bodies — reject anything else. - Reasonable request body size limits.

Handle: - Missing files. - Invalid JSON. - Invalid configuration keys. - Permission errors. - Backup failures. - Repair failures. - Unexpected exceptions. - Invalid identifiers. - Path traversal attempts.

Keep route handlers thin and place business logic in reusable Python service functions.

Do not expose arbitrary shell execution through the API.

Do not expose secrets or environment variables.

Security hardening

- **Never** run the Flask app with `debug=True` or the interactive Werkzeug debugger enabled, since the debugger allows arbitrary code execution if the endpoint is ever reachable. Use a `FLASK_ENV` / `FLASK_DEBUG` environment variable, defaulting to off.
- Load `SECRET_KEY` (if sessions or flash messages are used) from an environment variable or a local `.env` file

excluded from version control — never hardcode it in source.
- Apply basic rate limiting or at least a per-resource lock on state-changing endpoints (`/api/repair`, `/api/drift/simulate`, `/api/backups/<id>/restore`) so repeated or concurrent submissions cannot race or hammer the filesystem.
- Bind to `127.0.0.1` during development.
- Do not enable unrestricted CORS.
- If authentication is necessary for the deployment context, implement appropriate authentication, authorization and session security rather than inventing a fake login screen. For a purely local, single-user tool bound to `127.0.0.1`, explicitly state in the final deliverables whether authentication was implemented or intentionally omitted, and why.

PHASE 14: RESPONSIVE DESIGN

The dashboard must work on: - Desktop. - Laptop. - Tablet. - Mobile browser.

On smaller screens: - Collapse the sidebar into a navigation drawer. - Stack dashboard cards. - Keep charts readable. - Make tables scrollable or adapt them to mobile layouts. - Prevent horizontal page overflow. - Keep buttons accessible. - Use readable font sizes and appropriate touch targets.

Ensure all pages share the same design system and navigation.

PHASE 15: TESTING AND VERIFICATION

Before modifying anything, run:

```
python3 -m unittest discover -s tests -v
```

Preserve existing tests.

Add tests for: - Flask health endpoint. - Dashboard endpoint. - Configuration loading. - Drift scan. - Simulation validation. - Repair preview. - Repair allowlist enforcement (drifted-but-ineligible keys are reported, not repaired). - Backup creation before repair. - Successful repair verification. - Failed repair. - Backup listing. - Backup restore. - Report listing. - Audit log retrieval. - Invalid requests. - Invalid JSON. - Path traversal attempts (both backups and reports). - Missing files. - Concurrent repair/restore requests do not corrupt the configuration file.

Use temporary directories and test fixtures to avoid damaging project configuration during testing.

After implementation, run:

```
python3 -m unittest discover -s tests -v
```

Also manually verify: - All navigation links work. - All buttons work. - API requests return expected data. - The dashboard updates after scans and repairs. - Error states display correctly. - The CLI still works. - Existing functionality has not been broken. - No fake metrics remain in the UI. - No browser console errors occur during normal operation. - The app still works with no internet access (vendored assets load correctly).

Do not run destructive repair operations against production system configuration during tests.

PHASE 16: INSTALLATION AND RUNNING

Provide exact Linux commands to install dependencies and start the web dashboard.

Adapt commands to the actual files and environment.

The startup instructions should cover:

1. Opening the project directory.
2. Activating or creating the virtual environment.
3. Installing dependencies from the pinned `requirements.txt`.

4. Starting Flask (explicitly with debug mode off).
5. Opening the dashboard in a browser.
6. Stopping the server.
7. Running tests.

During development, the dashboard should normally be accessible at:

`http://127.0.0.1:5000`

Use this URL only if it matches the actual Flask configuration.

Do not expose the development server to the public internet. Note in the deliverables that the built-in Flask server is for development only, and that a production deployment (if ever needed) would require a WSGI server such as Gunicorn behind a reverse proxy — but do not implement that now, it is out of scope.

PHASE 17: FINAL DELIVERABLES

Complete the implementation and provide:

1. A working Flask backend.
2. A professional responsive web dashboard.
3. Configuration comparison.
4. Drift detection.
5. Drift simulation.
6. Safe automatic repair (with allowlist enforcement).
7. Backup management.
8. Report management.
9. Audit logs.
10. Responsive styling.
11. Updated tests.
12. Installation and startup instructions.
13. A pinned `requirements.txt` and `.gitignore`.
14. **A full list of created and modified files**, each with a one-line reason for the change.
15. Actual test results (pass/fail counts), not a claim that tests passed.
16. Known limitations, including an explicit statement on authentication status (implemented vs. intentionally omitted for local-only use).

Do not stop after creating a mockup.

Do not leave buttons unimplemented.

Do not replace real data with hardcoded sample data.

Do not claim tests passed unless they actually passed.

Do not delete the existing CLI.

FINAL EXECUTION INSTRUCTIONS

Start by inspecting the existing project files.

Then work incrementally:

1. Analyze the codebase.
2. Run the existing tests.
3. Create the shared service layer.
4. Implement the Flask backend (with security hardening from Phase 13).
5. Build the dashboard layout.
6. Implement each page.
7. Connect every feature to the real backend.
8. Add validation, backups, allowlist enforcement, and error handling.

9. Test the complete application, including offline asset loading and concurrency.
10. Fix issues found during testing.
11. Provide exact startup instructions and the file-change manifest.

If an operation could modify or damage configuration data, implement a safe preview, backup, validation and confirmation workflow.

Prioritize real functionality, correctness, security, maintainability and professional visual design.

Begin by inspecting the current project. Do not ask me to manually recreate files that already exist.